

Intelligent Classroom Energy Monitoring System

Prototype Edition: Real-Time Electricity Sensing, Forecasting & Anomaly Detection

Project code	BCAIAA22
Faculty guide	Dr. Tushar Mote
Team	Aditya Bade (Data + AI + Backend) Aditya Nalge Aryan Dhondage Om Teke
Version	October 2026 - revised, simplified for a bench-scale hardware prototype

1. What This Project Does (Simplified)

A small bench model of a classroom uses a **small DC fan** (classroom fan), an **LED bank** (classroom lights) and a **steady load** (projector). Each one is wired through its own current/power sensor. An ESP32 reads voltage, current and power every few seconds, adds context (brightness, temperature, class schedule) and sends everything to a backend. The backend stores it as **our own dataset** and an AI engine **predicts future consumption in real time**, flags abnormal usage, and tells which appliance is responsible.

What changed from the earlier document

Earlier version	This version
Trained on the UCI household dataset (not classroom data)	Own dataset generated by the bench setup; UCI used only to test the pipeline before data exists
Real fan / light / projector on a classroom circuit (PZEM, CT clamps, mains)	Small DC fan + LEDs + steady load on 12 V DC: safe, cheap, same data shape
Long EDA on household data (Belgium house, 29 columns)	Removed. Only the useful parts kept: feature engineering, chronological split, XGBoost/LSTM, residual anomaly method, FastAPI, DB
Appliance attribution was untestable	Now testable because each appliance has its own sensor
AI = forecast + anomaly	AI = forecast + anomaly + attribution + appliance disaggregation + waste detection + auto-advice (Section 5)

Important correction: the LDR (light sensor) measures **brightness**, not electricity. It is used as a context feature and to cross-check that a lit LED really draws power. **Electricity is measured by the INA219 current/power sensors** (voltage x current = watts). The INA219 is not in the lab list and must be bought (about 3 pieces).

2. Classroom to Bench Mapping

Real classroom	Bench stand-in	How it is measured	Controlled by
Ceiling lights	LED bank (white LEDs + resistors, or 12 V LED strip)	INA219 #1 (address 0x40)	MOSFET + PWM (dimming levels)
Ceiling fan	12 V DC fan (lab item 115)	INA219 #2 (address 0x41)	MOSFET + PWM (speed levels)
Projector	Second 12 V DC fan or 12 V 5 W bulb (steady, higher load)	INA219 #3 (address 0x44)	Relay (ON/OFF)
Class timetable	Virtual schedule flag (class / no class)	Software	Scenario engine on the backend
Daylight / room brightness	LDR module	Analog input (ESP32 ADC1)	-
Room climate	DHT11	Digital	-

Because the bench uses watts instead of kilowatts, the dashboard supports a **scale_factor** (for example x100) so bench watts can be shown as 'equivalent classroom' kWh and rupees. The AI always trains on raw bench watts; scaling is display-only.

3. Hardware Design

3.1 System architecture

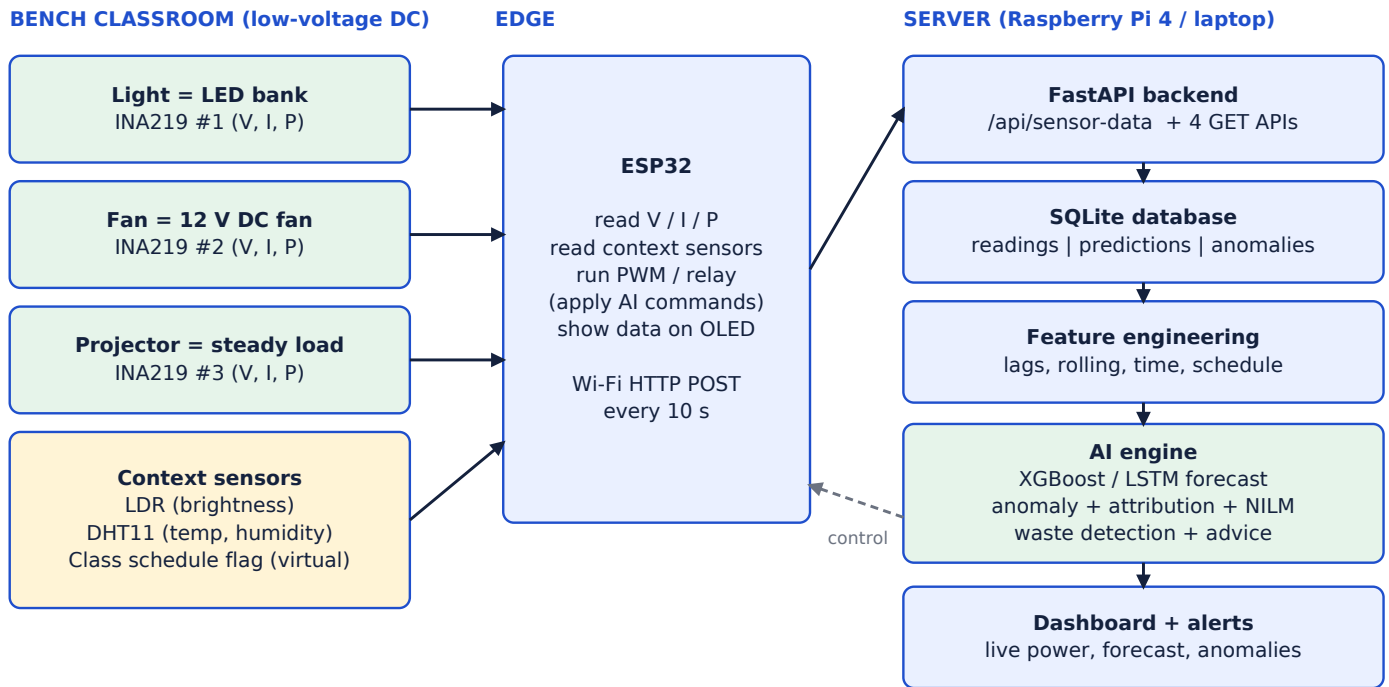


Figure 1: Data flows left to right; the AI engine can send a control command back (for example cut off the projector when the room is empty).

3.2 Components: what you already have vs what to buy

Part	Lab list item	Qty	Role
ESP32 microcontroller	#44 (qty 2)	1 (+1 spare)	Reads sensors, drives loads, Wi-Fi POST
Raspberry Pi 4 (or laptop)	#37 / #51	1	Runs FastAPI + database + AI model
12 V DC fan	#115 (qty 2)	1-2	Fan (and optionally projector stand-in)
LED assortment kit + 1k resistors	#36, #104	1 set	Light load (LED bank)
IRLZ44N N-channel MOSFET	#114 (qty 2)	2	PWM switching for light and fan
2-channel 5 V relay module	#26 / #129	1	Projector ON/OFF + spare channel for master cut-off
LDR sensor	#35	1	Ambient brightness (lux proxy)
DHT11	#7 / #108	1	Temperature + humidity
OLED 0.96 in	#9 / #61	1	Local live display (I2C)
12 V 5 A power supply	#42	1	Powers all loads
Breadboard, jumper wires	#31, #103	-	Wiring
Digital multimeter	#59	1	Calibrate INA219 readings
INA219 current/power sensor	NOT in list - buy	3	Measures V, I, P for each appliance (I2C)
12 V LED strip or 5 W bulb (optional)	NOT in list	1	Stronger, easier-to-read current than tiny LEDs
LM2596 buck module (optional)	NOT in list	1	12 V to 5 V for ESP32 (or just power ESP32 by USB)

3.3 ESP32 pin map

Signal	ESP32 pin	Notes
I2C SDA / SCL	GPIO21 / GPIO22	Shared by 3x INA219 (0x40, 0x41, 0x44) and OLED (0x3C)
Light MOSFET gate (PWM)	GPIO25	IRLZ44N is logic-level, 3.3 V gate drive works
Fan MOSFET gate (PWM)	GPIO26	Add flyback diode across the fan
Projector relay IN1	GPIO23	Relay ch2 (GPIO13) kept for master cut-off

Signal	ESP32 pin	Notes
LDR analog out	GPIO34	ADC1 pin (works while Wi-Fi is on)
DHT11 data	GPIO4	-

3.4 Wiring and safety rules

- **High-side sensing:** 12 V supply + goes to INA219 VIN+, INA219 VIN- goes to the load +, the load - goes to the MOSFET drain, MOSFET source to common GND.
- Set INA219 addresses with the A0/A1 solder jumpers (GND/GND = 0x40, VS+ = 0x41, SDA = 0x44). Use **one common ground** for supply, ESP32 and sensors.
- Do not measure current with the ESP32 ADC: it is noisy and non-linear. Use the INA219; average 16 samples per reading because PWM makes current ripple.
- Calibrate: compare INA219 current with the multimeter at 3 load levels and store a correction factor per channel.
- **Keep the prototype at 12 V DC.** Do not connect 230 V mains to breadboards. Real-classroom scale-up needs isolated meters (PZEM-004T / CT clamp) and supervision (Section 9).

4. Building Our Own Dataset

4.1 Scenario engine with time compression

A real classroom week would take 7 days to record. Instead the backend runs a **virtual timetable**: 1 real second = 1 virtual minute (60x). One virtual 10-minute step = 10 real seconds, so **one virtual day = 24 real minutes** and 60 virtual days (about 8,640 rows) is roughly 24 hours of running, easy to do in 2-3 overnight sessions. Each step the server sends the ESP32 a target state (light %, fan %, projector on/off) according to the virtual hour, weekday and class schedule.

Virtual situation	Light	Fan	Projector
Night / weekend / holiday	0%	0%	off
Class in session (lecture)	70-100%	60-100% (rises with heat)	on 60% of the time
Break / lab	50-70%	40-60%	off
Exam day	100%	80%	off
Hot afternoon	as class	100%	as class

Randomness (late start, extra class, early leave) is added so the AI cannot memorise the timetable. All time features and lags are computed on the **virtual clock** (lag_144 = 1 virtual day, lag_1008 = 1 virtual week); the real timestamp is also stored.

4.2 Labelled anomaly injection

About 3% of steps are deliberately faulty and labelled in the dataset. This lets us measure anomaly detection with real precision and recall:

- Fan left at full speed at night or on a holiday
- Projector left ON when no class is scheduled
- Lights ON in bright daylight with no class scheduled (LDR shows high brightness)
- Stuck / over-current load (power 2x normal) and sensor dropout (reading goes to 0)

4.2b Dataset columns

Column	Unit	Source
timestamp, virtual_time	ISO 8601	Real clock and virtual classroom clock
light_v, light_ma, light_w	V, mA, W	INA219 #1
fan_v, fan_ma, fan_w	V, mA, W	INA219 #2
proj_v, proj_ma, proj_w	V, mA, W	INA219 #3
total_w, energy_wh	W, Wh	Sum of appliances; energy over the virtual 10-min interval
lux_raw	ADC 0-4095	LDR
temp_c, humidity	C, %	DHT11
schedule_flag	0/1	Timetable (class in session)

Column	Unit	Source
is_injected_anomaly, fault_type	0/1, text	Scenario engine (ground truth labels)

5. AI Layer (Expanded)

5.1 AI modules

#	Module	What it does	Technique
1	Real-time forecast	Predicts total and per-appliance power for the next 10 min, next hour and next day	XGBoost (production), LSTM (comparison), baselines
2	Anomaly detection	Flags abnormal consumption given time and class schedule; severity in sigma	Residual +/-3 sigma (kept from earlier design) vs Isolation Forest
3	Appliance attribution	Names the culprit: light, fan or projector (largest residual)	Per-appliance residuals (now validated)
4	Load disaggregation (NILM)	Estimates each appliance from the total only ; proves scalability to a real classroom with one main meter	Multi-output XGBoost / small neural net trained on total power + context
5	Waste detection	Appliance ON when no class is scheduled, lights ON in bright daylight	Schedule-aware model + rules from learned baselines
6	Smart advice and auto-action	Suggests or triggers cut-off, estimates saving in kWh, Rs and CO2	Decision layer on forecasts; relay command to ESP32
7	Daily summary (optional)	Plain-language report of the day, anomalies and tips	LLM call on the day's aggregated stats

5.2 Features

Group	Features
Cyclical time (virtual)	hour_sin, hour_cos, dow_sin, dow_cos, is_weekend, schedule_flag
Lag power	lag_1, lag_2, lag_3, lag_6, lag_12, lag_144, lag_1008 for total_w (and per appliance)
Rolling	roll_mean_6, roll_mean_12, roll_std_6, roll_mean_144
Classroom context (new)	lux_raw, temp_c, humidity, minutes_since_class_start
Per-appliance (new)	light_w, fan_w, proj_w and their lags

5.3 Training and evaluation

- **Chronological split only** (70% train / 15% validation / 15% test). No random shuffling because lag features would leak the future.
- Models compared: Naive (next = current), Linear Regression, Random Forest, XGBoost (early stopping on validation), LSTM (12-step window, 2 layers).
- Forecast metrics: MAE, RMSE, R-squared, MAPE. Anomaly metrics: precision, recall, F1 using the injected labels. NILM metric: per-appliance MAE.
- Ablation test: forecast with and without **schedule_flag** and **lux_raw** to show the value of context features.
- The UCI results in the earlier document (LSTM R-squared 0.575, XGBoost MAE 34.14 Wh) came from household data and must **not** be quoted as classroom results. Report only numbers from our own dataset.

5.4 Anomaly logic

```

expected = model.predict(features)          # per appliance and total
residual = actual - expected
sigma_z  = (residual - mean_val) / std_val  # mean/std from validation set
anomaly  = abs(sigma_z) > 3
culprit  = argmax over appliances of abs(appliance_residual)
waste_flag = (appliance_power > idle_threshold) and (schedule_flag == 0)

```

6. Backend, Database and Dashboard

6.1 API (FastAPI)

Endpoint	Method	Purpose
/api/sensor-data	POST	ESP32 sends a reading; server stores it, predicts, checks anomaly, replies with a control command
/api/current	GET	Latest reading (live panel)

Endpoint	Method	Purpose
/api/history	GET	Aggregated history (hourly / daily) for charts
/api/prediction	GET	Next 10 min / hour / day forecast, total and per appliance
/api/anomalies	GET	Anomaly log (filter by appliance, severity)
/api/appliances (new)	GET	Per-appliance share of energy and waste events
/api/control (new)	POST	Manual override or enable/disable auto cut-off

Note: the earlier document used both /api/sensordata and /api/sensor-data. Use **/api/sensor-data** everywhere.

6.2 ESP32 payload and server reply

POST /api/sensor-data

```
{ "timestamp":"2026-10-12T14:30:10", "virtual_time":"2026-03-02T10:20:00",
  "light_w":0.46, "fan_w":1.92, "proj_w":3.24, "total_w":5.62,
  "energy_wh":0.937, "schedule_flag":1,
  "lux_raw":2150, "temp_c":27.3, "humidity":55 }
```

Reply:

```
{ "predicted_next_w":5.9, "predicted_next_hour_wh":35.4,
  "anomaly":false, "severity_sigma":0.4, "culprit":null,
  "command":{"light_pct":80,"fan_pct":70,"projector":1} }
```

6.3 Database tables (kept simple)

Table	Main columns
sensor_readings	id, timestamp, virtual_time, light_w, fan_w, proj_w, total_w, energy_wh, lux_raw, temp_c, humidity, schedule_flag, is_injected_anomaly, fault_type, created_at
predictions	id, for_time, horizon (10min / 1hour / 1day), predicted_total_w, predicted_light_w, predicted_fan_w, predicted_proj_w, model_version, created_at
anomalies	id, detected_at, actual_w, expected_w, residual_w, severity_sigma, anomaly_type, likely_appliance, is_confirmed, notes, created_at

6.4 Dashboard panels

- Live power gauge per appliance and total, with today's energy and cost
- Forecast curve (actual vs predicted, next hour and next day)
- Anomaly log with severity and culprit appliance, plus waste alerts
- Appliance share pie chart, saving suggestions, scale_factor setting for equivalent-classroom view

7. ESP32 Firmware Logic

```
setup(): init I2C, 3x INA219, OLED, DHT11, pins, PWM, Wi-Fi
loop() every 1 s (one virtual minute):
  read INA219 x3 (average 16 samples) -> light_w, fan_w, proj_w
  read LDR, DHT11
  accumulate into a 10 s window (= one virtual 10-minute step)
every 10 s:
  build JSON (mean power of window + context)
  HTTP POST /api/sensor-data
  parse reply.command -> set light PWM, fan PWM, projector relay
  show total W, forecast and anomaly status on OLED
on Wi-Fi loss: buffer up to 500 readings in RAM, resend when back
```

8. Full Build Structure

8.1 Project folders

```
classroom-energy/
|-- firmware/esp32_energy/esp32_energy.ino      # sensors, PWM, POST, OLED
|-- backend/
|   |-- main.py                                # FastAPI entry
|   |-- api/ (sensor.py, energy.py, prediction.py, anomaly.py, appliances.py, control.py)
|   |-- scenario.py                            # virtual timetable + fault injection
|   |-- schemas.py, db.py
|-- ml/
|   |-- preprocessing.py                       # features (shared by train + live)
|   |-- train.py                               # all models, chronological split
|   |-- predict.py, anomaly.py, nilm.py, advice.py
|   |-- models/ (xgboost_v1.json, lstm_v1.keras, thresholds.npy)
|-- data/ (schema.sql, classroom_energy.db, exports/*.csv)
|-- dashboard/ (index.html, Chart.js)
```

| -- docs/ (wiring diagram, calibration sheet, this document)

8.2 Step-by-step roadmap

Phase	Work	Owner	Output / check
0 - Buy and prepare	Buy 3x INA219 (+ LED strip / buck module if wanted); collect lab parts	Hardware team	Parts on desk
1 - Sensing	Wire one INA219 + LED + MOSFET; read V/I/P over serial; calibrate against multimeter	Hardware team	Error under 5% vs multimeter
2 - All loads	Add fan and projector channels, LDR, DHT11, OLED	Hardware team	All 3 appliances + context printing correctly
3 - Backend	Run FastAPI + SQLite; ESP32 POSTs real readings; dashboard live panel	Aditya Bade + team	Rows appear in sensor_readings every 10 s
4 - Data collection	Run scenario engine with time compression; inject labelled faults; export CSV	All	At least 30 virtual days (~4,300 rows), target 60
5 - AI training	Feature engineering, chronological split, train all models, anomaly thresholds, NILM	Aditya Bade	Comparison table on our own data
6 - Live AI	Load best model in predict.py; real-time forecast + anomaly + advice in the API reply	Aditya Bade	Forecast and alerts on the dashboard
7 - Closed loop and demo	Enable auto cut-off on waste; live demo (projector left on outside the schedule)	All	Anomaly detected and appliance named on stage
8 - Report	Results, ablation, limitations, scale-up plan	All	Final capstone report

8.3 Testing checklist

- Each INA219 matches the multimeter at 3 load levels; correction factor stored.
- Data rows are continuous: no gaps, no duplicates, timestamps and virtual_time consistent.
- A fault injected by the scenario engine shows up in anomalies with the correct appliance.
- Forecast beats the naive baseline on the test split; improvement from schedule and brightness features is reported.
- System keeps running 24 h without crash; Wi-Fi drop test recovers with buffered data.

9. Limitations and Scale-Up

Limitation	Impact	Plan
Bench loads are tiny and simplified	Fan and projector signatures differ from real ones	Use scale_factor for display; retrain on real data when available
Time-compressed data is partly simulated (virtual schedule)	Patterns reflect our timetable assumptions	Document assumptions; add randomness; validate on a short real-time run
No mains measurement	Not yet usable in a real room as is	Replace INA219 with PZEM-004T / CT clamps (isolated); same JSON payload, same AI pipeline
No occupancy sensing (by design)	Cannot tell directly if the room is empty	Uses the class timetable and LDR brightness instead; people detection can be added later if ever needed
Model drift (season, timetable change)	Accuracy drops over time	Retrain monthly; keep versions (do not overwrite v1)

Why AI instead of a fixed threshold: consumption legitimately varies several times between an empty night and a full lecture. A rule such as 'alert above X watts' either misses waste at night or fires during every class. The model learns what is normal for this hour, weekday, schedule and time of day, so the threshold adapts, and it can also say which appliance caused the problem and what will happen next.